Faculty of Engineering and

Technology Department of

Computer Science

COMP338-Artifical Intelligence

2026

Genetic Algorithms

Instructor Name:

Radi Jarrar

Group Members:

Nahed Najjar -1220704

Sylia Darabuzedan -1230838

Contents

Introduction:

Universities have the problem of scheduling exams every semester. The aim is to schedule exams in available time slots, under rules on students and courses. Poor scheduling can cause students to have exams at the same time, or students to have too much work at the same time. In this project we consider 95 students taking 22 courses. We want to make a working exam schedule without any clashes. There are a whole lot of potential schedules." It's not possible to check all options to find the best one. Therefore, we use a Genetic Algorithm. This method is based on the way nature works and it is good for solving scheduling problems like this. The main objective of our work is to design a Genetic Algorithm that generates exam timetables that respect the rules with the smallest number of exam days and minimum conflict, so that the timetable is fair for the students.

.

Background of Genetic Algorithm:

A Genetic Algorithm is a population-based optimization technique inspired by the principles of natural selection and genetics. It works by evolving a population of candidate solutions using selection, crossover and mutation to find optimal or near optimal solutions to complex problems.

Steps:

1. **Initialization:** We create a group of random exam schedules to start with. Each schedule represents a possible answer to the problem of making a good exam timetable.
2. **Evaluation:** We assess each schedule and assign it a fitness score depending on how well it meets the requirements. The fitness is calculated by a penalty-based formula, in which the more the number of constraint violations in the schedule, the higher the penalty and the lower the fitness: fitness = 1.0 / (1 + penalty)
3. **Selection**: We pick schedules to be parents. Schedules with higher fitness scores have a better chance of being chosen.
4. **Crossover**: We create a new schedule by blending two parent schedules. We use the beginning from one and the end from the other. So, the new schedule gets good traits from both parents.
5. **Mutation**: We make small random changes to the new schedules with a low probability. This keeps things diverse and stops the algorithm from getting stuck in a rut at a suboptimal solution.
6. **Replacement:** The new schedules replace the old ones to create the next generation. Yet, we hold on to the best chromosome from the current round using elitism to ensure the solution doesn't slide backwards.
7. **Termination:** The process wraps up once the max generations limit is hit or when the fitness level peaks at 1.0 signaling a perfect schedule.

Setting Parameters:

1- **Population size:** The number of schedules we keep at each step.

2- **Mutation probability:** How often we make random changes to the schedules.

3- **Max Generation:** How many rounds we run.

Problem Formulation

Problem Definition

The problem is to create an exam schedule for 95 students taking 22 courses. We use a Genetic Algorithm to evolve potential schedules over time, reducing rule breaches along the way. Eventually, we find the optimal schedule that works best.

Formulation of the Problem:

- **Initial State: Initial State:** A randomly generated population of exam schedules, each containing 22 genes (one per course) with randomly assigned time slots.
- **State:** Each state is a set of chromosomes (exam schedules) in the current generation.
- **Action:** Actions include selecting parent schedules, applying crossover, and introducing mutations to create new schedules.
- **Transition Model:** The new population is generated by applying the Genetic Algorithm operators (selection, crossover, and mutation) to the current population.
- **Goal Test:** The goal is reached when a chromosome has a fitness of 1.0 meaning no constraint violations exist in the schedule.
- **Path Cost:** The cost is measured by the number of generations required and the number of penalties in the best schedule found.

Representation of Chromosome & Gene:

Gene Representation: Each gene represents one course and its assigned time slot.

```
public Gene(String Course_Code, String Slot_ID) {
    this.Course_Code = Course_Code;
    this.Slot_ID = Slot_ID;
}
```

Example:

Gene = (COMP211, D1S1).

Chromosome Representation: A chromosome is a complete exam timetable. Since there are 22 courses, each chromosome has 22 genes.

```
public Chromosome() {
    genes = new ArrayList<>()
    fitness = 0;
}
```

Example:

Chromosome = [(COMP2110, D1S2), (COMP2340, D1S1), ….].

fitness function:

The fitness is calculated by counting constraint violations (penalties).

1. student should not have two exams at the same slot

```
// Same slot
if (slot1.equals(slot2)) {
    penalty += 1000;
}
```

2. student should not have two exams in the same day

```
// same day
if (getDay(slot1).equals(getDay(slot2)) && !slot1.equals(slot2)) {
    penalty += 50;
}
```

3. A student should not have more than two exams in the same day

```
// no student has more than two exams in the same day
for (String day : examsPerDay.keySet()) {
    int count = examsPerDay.get(day);
    if (count > 2) {
        penalty += (count - 2) * 800;
    }
}
```

4. A student should not have four exams in two days in a row

```
// student should not have four exams over two consecutive exam days.
for (int day = 1; day <= 20; day++) {
    int current = examsPerDay.containsKey("D" + day) ? examsPerDay.get("D" + day) : 0;
    int next = examsPerDay.containsKey("D" + (day + 1)) ? examsPerDay.get("D" + (day + 1)) : 0;
    if (current + next >= 4) {
        penalty += 500;
    }
}
```

5. minimum number of exam days

```
// Preferred max used days-->5
if (slotsUsedPerDay.size() > 5) {
    penalty += (slotsUsedPerDay.size() - 5) * 100;
}
```

6. A day can only fit three exam time slots

```
// day has at most three exam slots.
for (Map.Entry<String, Integer> entry : slotsUsedPerDay.entrySet()) {
    if (entry.getValue() > 3) {
        penalty += (entry.getValue() - 3) * 800;
    }
}
```

The fitness is calculated as:

```
    return 1.0 / (1 + penalty);
}
```

This means a schedule with zero penalties has a fitness of 1.0, and the more penalties, the closer the fitness gets to 0.

Implementation of Genetic Algorithms:

1. Initialization: First, create a random exam schedule. That is, assign a random time slot for each of the 22 courses. Then it generates an initial population of schedules by calling the RandomChromosome method. Any schedule is a feasible solution.

```
private Chromosome RandomChromosome(List<String> courses, List<String> slots) {
    Chromosome chromosome = new Chromosome();

    for (String course : courses) {
        //random Slot
        String randomSlot = slots.get(random.nextInt(slots.size()));
        chromosome.addGene(new Gene(course, randomSlot));
    }
    return chromosome;
}
```

It then repeatedly calls the RandomChromosome method to generate an initial population of exam schedules. Each schedule is a candidate solution in which each course is assigned to a time slot randomly.

```
public void Population(int size, List<String> courses, List<String> slots) {
    for (int i = 0; i < size; i++) {
        chromosomes.add(RandomChromosome(courses, slots));
    }
}
```

2. Evaluation: Calculates the fitness of each chromosome by counting constraint penalties. The more violations a schedule has, the higher the penalty, and the lower the fitness.

```
private void initializePopulation() {
    for (Chromosome c : population.getChromosomes()) {

        c.setFitness(fitnessFunction.calculateFitness(c));
    }
}
```

3. Selection: Choose individuals based on their fitness to form the next generation. Chooses one chromosome to be a parent based on its fitness using. Higher fitness chromosomes have a higher probability of being selected.

```java
public Chromosome select(List<Chromosome> chromosomes) {
    double totalFitness = 0;
    for (Chromosome c : chromosomes) {
        totalFitness += c.getFitness();
    }
    double rand = random.nextDouble() * totalFitness;
    double Sum = 0;
    //Sum of fitness-->min Number of conflict
    for (Chromosome c : chromosomes) {
        Sum += c.getFitness();
        if (Sum >= rand) {
            return c;
        }
    }
    return chromosomes.get(chromosomes.size() - 1);
}
```

4. Crossover: Combines two parent schedules at a random crossover point to produce a new child schedule. The child takes the first part from parent1 and the second part from parent2.

```java
//one-point
    public Chromosome crossover(Chromosome parent1, Chromosome parent2) {
        Chromosome child = new Chromosome();
        int crossoverPoint = random.nextInt(parent1.size());

        for (int i = 0; i < parent1.size(); i++) {
            if (i < crossoverPoint) {
                child.addGene(new Gene(
                    parent1.getGenes().get(i).getCourse_Code(),
                    parent1.getGenes().get(i).getSlot_ID()
                ));
            } else {
                child.addGene(new Gene(
                    parent2.getGenes().get(i).getCourse_Code(),
                    parent2.getGenes().get(i).getSlot_ID()
                ));
            }
        }
        return child;
    }
```

5. Mutation: Introduce random changes to maintain diversity.

```java
    public void mutate(Chromosome chromosome, List<String> slots, double mutationRate) {
        for (int i = 0; i < chromosome.size(); i++) {
            if (random.nextDouble() < mutationRate) {
                String newSlot = slots.get(random.nextInt(slots.size()));
                chromosome.getGenes().get(i).setSlot_ID(newSlot);
            }
        }
    }
```

Randomly changes the time slot of a course with a small probability (mutation rate). This prevents the algorithm from getting stuck at a local solution.

6. ## Termination:

It creates a new population of schedules by combining and mutating the best schedules of the current population. The algorithm terminates when:

Maximum number of generations reached

The fitness reaches 1.0 (perfect schedule without any violations)

```
    public Chromosome run() {
        Chromosome best = getBestChromosome();

        for (int gen = 0; gen < generations; gen++) {
            List<Chromosome> newGeneration = new ArrayList<>();
            newGeneration.add(best);

            while (newGeneration.size() < populationSize) {
                Chromosome parent1 = selection.select(population.getChromosomes());
                Chromosome parent2 = selection.select(population.getChromosomes());

                Chromosome child = operators.crossover(parent1, parent2);
                operators.mutate(child, data.getSlots(), mutationRate);
                child.setFitness(fitnessFunction.calculateFitness(child));

                newGeneration.add(child);
            }

            population.getChromosomes().clear();
            population.getChromosomes().addAll(newGeneration);

            best = getBestChromosome();
            convergence.add(best.getFitness());

//Stop
            if (best.getFitness() >= 1.0) {
                break;
            }
        }

        return best;
    }
```

Parameter Tuning and Analysis

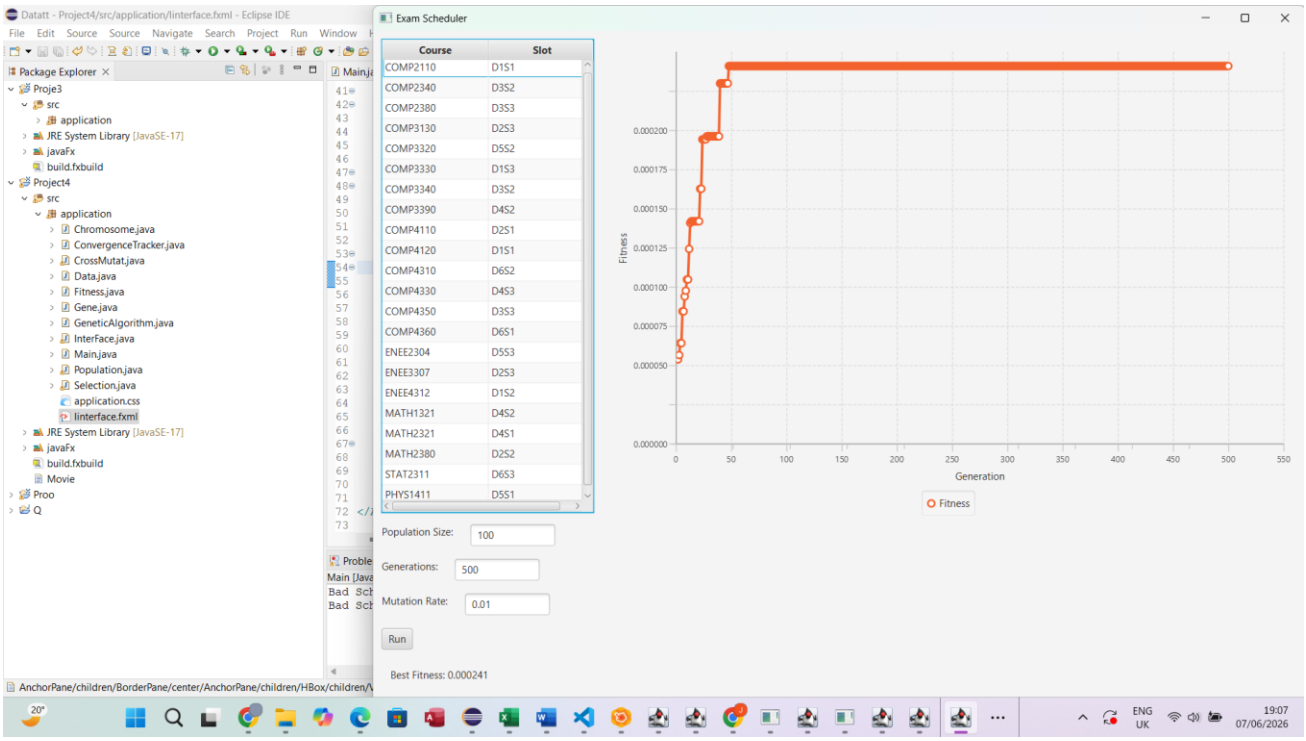
The algorithm was tested with varying population sizes, mutation rates, and maximum generations to determine their effects on convergence.

Results and Discussion

First Attempt

Population Size: 100, Generations: 500, Mutation Rate: 0.01

In this attempt, involved using a small population size and low mutation rate. Improvement was experienced up to generation 50 after which there was stabilization at 0.000241 fitness levels. The low mutation rate ensured that good solutions were retained without making any changes randomly.

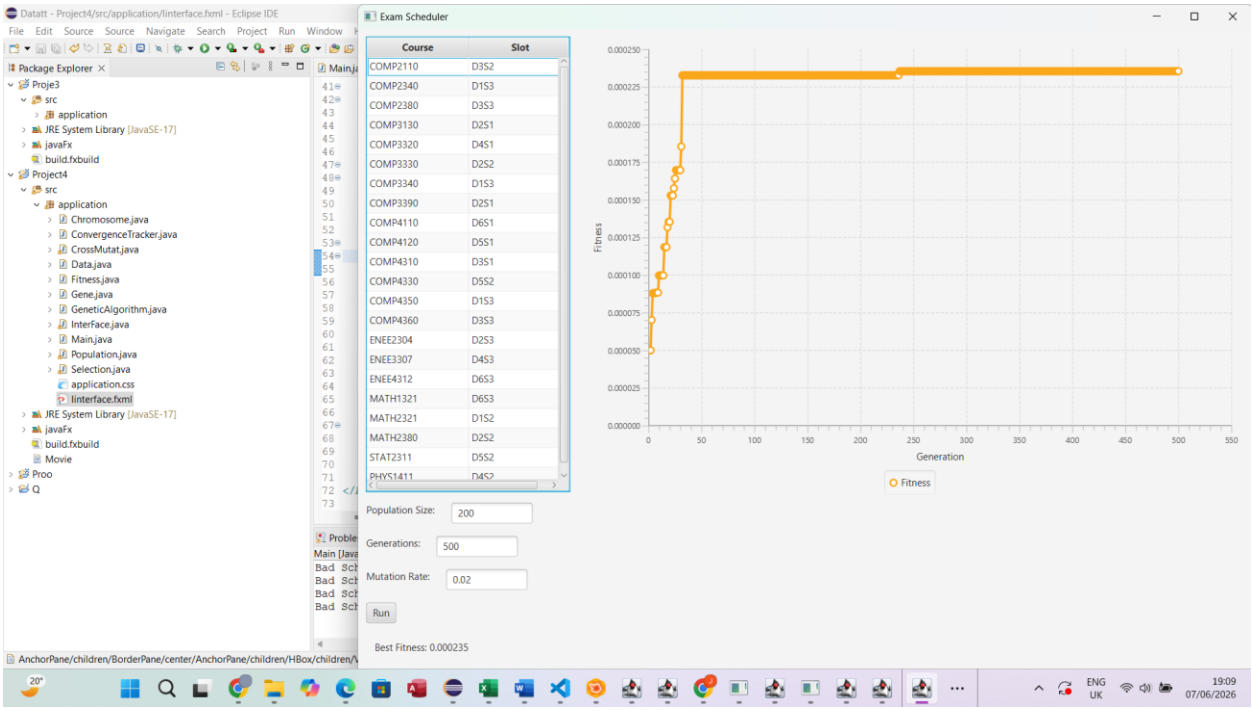


Second Attempt

Population Size: 200, Generations: 500, Mutation Rate: 0.02

Increasing the population size and mutation rate led to a small improvement in the best fitness value (0.000235). The algorithm converged quickly within the first 40 generations and then showed little change. This suggests that the higher mutation rate helped explore more solutions while still maintaining stable convergence.

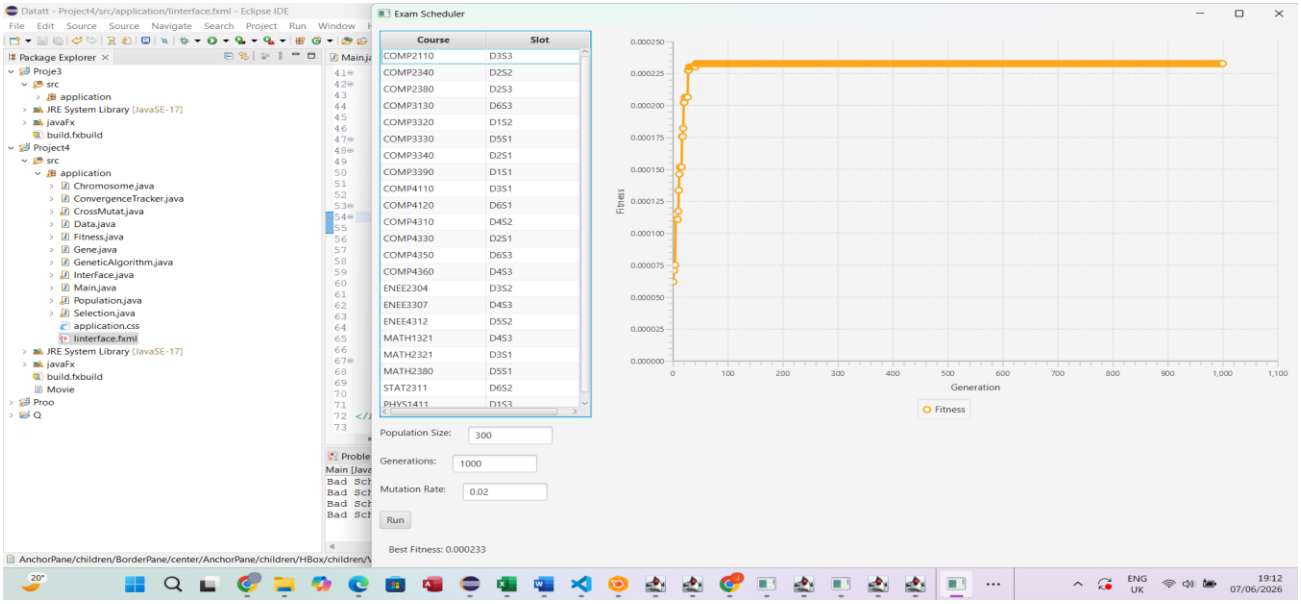
.



Third Attempt

Population Size: 300, Generations: 1000, Mutation Rate: 0.02

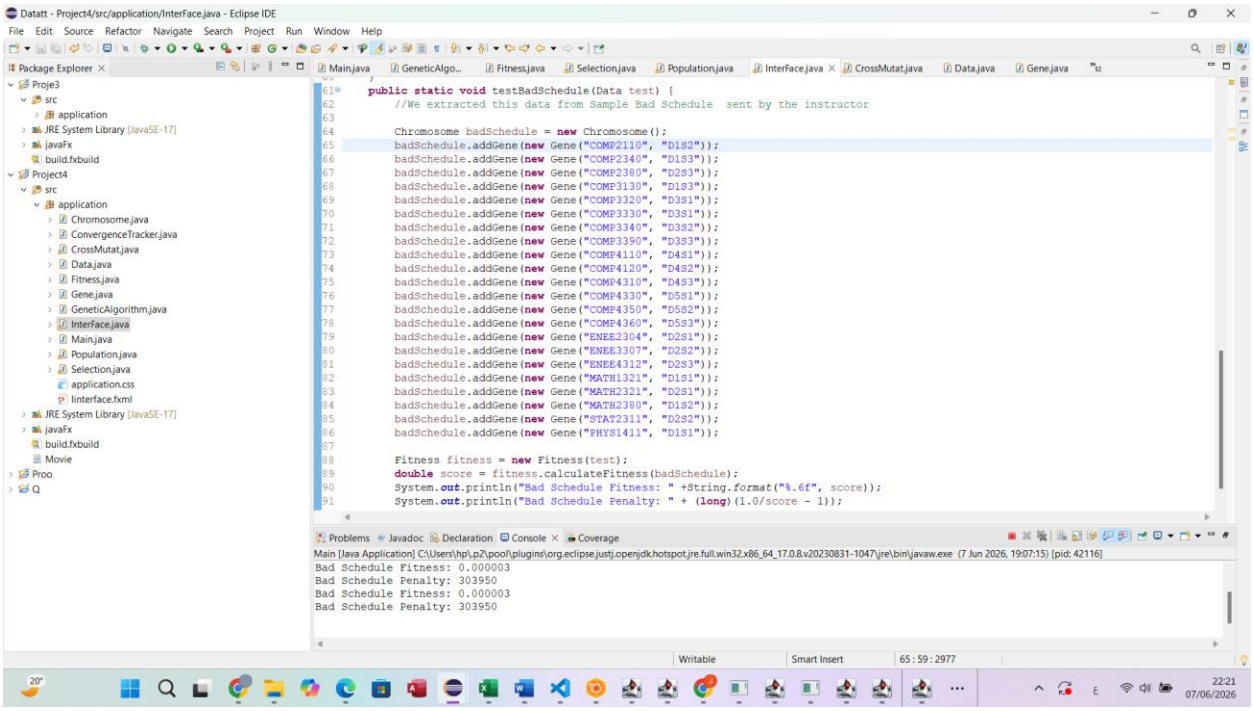
The results showed that using the largest population size with a larger number of generations did not achieve additional improvement, as the best fitness value remained at 0.000233. The algorithm converged during the first 20 generations and the results stabilized after that, showing that increasing the number of generations after 50 had little effect on the quality of the final solution.



All hard constraints were satisfied: no student has two exams at the same time, no student has more than two exams per day, and no student has four exams in two consecutive days.

Test fitness function

To test the effectiveness of our fitness function, it was applied to the example bad schedule provided by Instructor in the dataset. The result for the example bad schedule was a fitness score of 0.000003 (penalty = 303,950).



Conclusion

Larger population sizes improved the algorithm's accuracy but increased computation time. Higher mutation rates prevented premature convergence but disrupted convergence at excessively high values. The convergence of the algorithm was rapid at an early stage (between 20 to 50 based on the parameter settings) and then it stabilized without much improvement thereafter.

References

1. Course slides - COMP338 Artificial Intelligence, Birzeit University, 2026.
2. JavaFX documentation
3. Artificial Intelligence: A Modern Approach 3rd edition. Russell and Norvig, Pearson, 2010
4. GeeksforGeeks-Genetic Algorithms.
5. JavaFX Scene Builder Documentation